Complexity Classes

CS1200: Introduction to Algorithms and their Limitations
Lecture 19

Fall 2026

Harvard SEAS

Put away your devices

Announcements

- Reading: Hesterberg–Vadhan Chapter 19; Section 20.1 for the last slide.

Problems, not algorithms

From comparing algorithms to comparing problems

So far: comparing algorithms for a single problem.

Sorting		Graph 3-Coloring	
ExhaustiveSearchSort	$O(n! \cdot n)$	exhaustive search	$O(m \cdot 3^n)$
InsertionSort	$O(n^2)$	3by2Coloring (SRE 2)	$O(1.89^n)$
MergeSort	$O(n \log n)$		

Now: in this unit we compare the hardness of problems.

- How can we show a problem is easy? Find an algorithm.
- How can we show a problem is hard? Rule out *all* algorithms?

The complexity of a problem $\approx$ the runtime of the best algorithm for it.

Search and decision problems

Computational problem $\Pi = (\mathcal{I}, \mathcal{O}, f)$: on input x, the valid outputs are $f(x)$.

- **Search problem:** output any $y \in f(x)$, or $\perp$ if $f(x) = \emptyset$.
- **Decision problem:** yes or no.

search	decision
find a schedule of $\geq k$ intervals	is there a schedule of $\geq k$ intervals?
find a satisfying assignment	is φ satisfiable?

Every computational problem is solved search-style. A decision problem is the special case where

- outputs are restricted: $\mathcal{O} = \{\text{yes}, \text{no}\}$;
- each input has exactly one valid output, so $f(x) \neq \emptyset$ and the answer is never $\perp$.

In groups: problems we've seen

For each *problem*: one algorithm we've seen, and the best worst-case runtime we've seen.

problem	type	an algorithm we've seen	best worst-case runtime we've seen
Sorting	search	MergeSort	$O(n \log n)$
Sorting on Finite Universe	search	SingletonBucketSort	$O(n + U)$
Interval Scheduling–Decision	decision	sort, then scan	$O(n \log n)$
Shortest Path	search	BFS	$O(n + m)$
Graph 2-Coloring	search	BFS, color by layer	$O(n + m)$
Bipartite Max. Matching	search	augmenting paths	$O(nm)$
2-SAT	search	resolution	$O(n^4)$
Graph 3-Coloring	search	3by2Coloring (SRE 2)	$O(1.89^n)$
SAT	search	exhaustive search, resolution	$O(2^n \cdot \lvert\varphi\rvert)$
Long Path	search	exhaustive search over paths	$O(k \cdot n^k)$
Independent Set (max)	search	exhaustive search	$O(2^n(n + m))$

Linear, nearly linear, polynomial, exponential. Are the last four problems exponential, or have we just not found better algorithms?

$\mathsf{P}_{\text{search}}$ and $\mathsf{EXP}_{\text{search}}$

Two conventions for calculating problem size and runtime

1. One size measure for every problem.

- Input: an array of N cells, each holding a number $< \mathrm{poly}(N)$, i.e. $O(\log N)$ bits.
- Size $|x| = N$.
- Graph: $N = \Theta(n + m)$ (vertex names $< n$)
- k-CNF formula: $N = O(n + km)$ (variable indices $< n$)
- A big number is split across many cells.

2. Polynomial slack.

- Only distinguish polynomial from not.
- Extended Church–Turing: models differ by at most a polynomial slowdown.
- So any reasonable size measure ($n + m$, N, bits) gives the same classes.
- So we state runtimes in natural parameters (n, m, k): polynomial in them = polynomial in N.

Definition 19.1

$\mathsf{TIME}_{\text{search}}(T(n))$: problems solvable by a Word-RAM program in time $O(T(n))$ on inputs of length n (entries of bitlength $O(\log n)$).

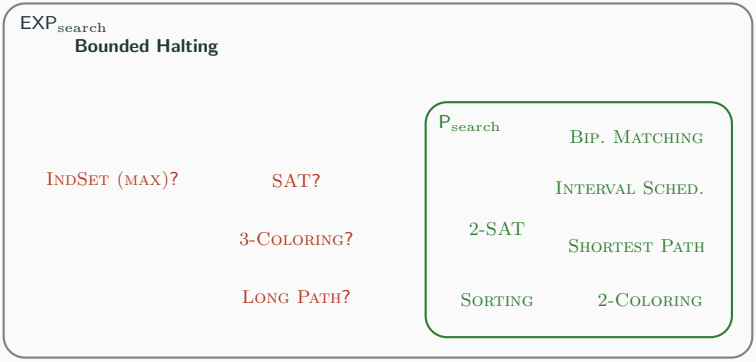
$$\mathsf{P}_{\text{search}} = \bigcup_{c \geq 0} \mathsf{TIME}_{\text{search}}(n^c) \qquad \mathsf{EXP}_{\text{search}} = \bigcup_{c \geq 0} \mathsf{TIME}_{\text{search}}\left(2^{n^c}\right)$$

$\mathsf{TIME}_{\text{search}}(T(n))$ is the set of problems that some algorithm solves in time $O(T(n))$.

A problem solvable in time $O(n^{\log n})$:

- in $\mathsf{P}_{\text{search}}$? Not from this algorithm: $\log n$ is not a constant.
- in $\mathsf{EXP}_{\text{search}}$? Yes: $n^{\log n} = \left(2^{\log n}\right)^{\log n} = 2^{(\log n)^2} \leq 2^n$ for large n, so it is in $\mathsf{TIME}_{\text{search}}(2^{n^1})$.

What we know



Theorem 19.2. $\mathsf{P}_{\text{search}} \subsetneq \mathsf{EXP}_{\text{search}}$. Example: Bounded Halting (Unit 5).

$\mathsf{NP}_{\text{search}}$

Can you find a cat?



Can you verify that the cat is in the circle?



Finding vs. checking

Definition 19.3

$\Pi = (\mathcal{I}, \mathcal{O}, f)$ is in $\mathrm{NP}_{\mathrm{search}}$ if

1. **all valid outputs are short:** there is a polynomial p with $|y| \leq p(|x|)$ and every $y[i] < p(|x|)$, for every input x and every $y \in f(x)$;
2. **all valid outputs are checkable:** a polynomial-time verifier V, given the input x and any y, decides whether $y \in f(x)$.

For problems in $\mathrm{NP}_{\mathrm{search}}$: finding a solution may be hard, but recognizing one is easy.

Check: why does Long Path satisfy condition 1? A path has at most n entries, each a vertex name $< n$.

SAT $\in \mathrm{NP}_{\mathrm{search}}$ (Proposition 19.4)

- $\mathcal{I}$: formulas $\varphi(x_0, \ldots, x_{n-1})$
- $\mathcal{O}$: assignments $\alpha \in \{0,1\}^n$
- $f(\varphi)$: satisfying assignments, $\{\alpha \in \{0,1\}^n : \varphi(\alpha) = 1\}$

1. **Short:** α is n bits, a truth value for each of the n variables. $|\alpha| = n \leq |\varphi|$

2. **Verifier** $V(\varphi, \alpha)$:
   - check that α is a string of n bits; otherwise reject
   - substitute α into φ and evaluate
   - time: $O(|\varphi|)$

In groups: Graph Coloring $\in \mathrm{NP}_{\mathrm{search}}$

Input: a graph $G = (V, E)$ and a number k.

- $f(G, k) = \{c : V \to [k] \text{ with } c(u) \neq c(v) \text{ for every edge } \{u, v\}\}$

1. **Short:** c is n cells, each holding a color $< k \leq n$
2. **Verifier** $V(G, k, c)$:
   - check c has n entries, each in $[k]$
   - check $c(u) \neq c(v)$ for every edge $\{u, v\}$
   - time: $O(n + m)$

Same for Independent Set–Threshold: $f(G, k) = \{S : S \text{ independent}, |S| \geq k\}$.

Threshold vs. maximum

problem	valid outputs	verifier?	in $\mathrm{NP}_{\mathrm{search}}$?
IndSet–Threshold	independent S, $\lvert S\rvert \geq k$	check pairs, count	yes
IndSet–Optimization	independent S of *maximum* size	"no bigger one"? no fast check known	not known
Maximum Matching	matchings of maximum size	no augmenting path (one BFS)	yes

Checking a solution is valid: usually easy.
Checking that nothing better exists: the hard part.

$\mathrm{NP}_{\mathrm{search}} \subseteq \mathrm{EXP}_{\mathrm{search}}$ **(Proposition 19.5)**

ExhaustiveSearch(x)

```
0  for all y with |y| ≤ p(|x|), y[i] < p(|x|):
1      if V(x, y) accepts: return y
2  return ⊥
```

Correct because *every* valid output is short.

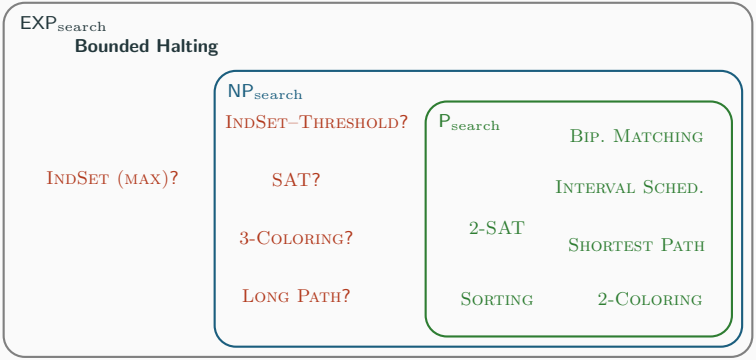
Candidates: at most $p(n)^{p(n)}$

$p(n)^{p(n)} = 2^{\log_2 p(n) \cdot p(n)} \leq 2^{n^d}$

Total: $2^{n^d} \cdot \mathrm{poly}(n) = 2^{\mathrm{poly}(n)}$.

And $\mathrm{P}_{\mathrm{search}}$ vs. $\mathrm{NP}_{\mathrm{search}}$? Strictly speaking, $\mathrm{P}_{\mathrm{search}} \not\subseteq \mathrm{NP}_{\mathrm{search}}$ (as you will show on PS7). But the natural problems in $\mathrm{P}_{\mathrm{search}}$ are in $\mathrm{NP}_{\mathrm{search}}$. For decision problems, $\mathrm{P} \subseteq \mathrm{NP}$ (Lemma 21.5).

The picture now



Problems can move: Maximum Matching on general graphs (1965), Linear Programming (1979), Primality Testing (2002).

Open: is $\mathsf{NP}_{\text{search}} \subseteq \mathsf{P}_{\text{search}}$? (the P vs. NP question, Ch 21)

Reductions

The hardest problems in $\mathrm{NP}_{\mathrm{search}}$

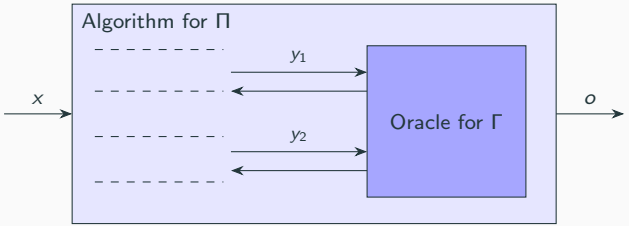
Surprising fact: SAT, 3-Coloring, Long Path, IndSet–Threshold, and thousands more are linked: if any one of them is in $\mathrm{P}_{\mathrm{search}}$, they all are.

Γ is $\mathrm{NP}_{\mathrm{search}}$-**complete** if

1. $\Gamma \in \mathrm{NP}_{\mathrm{search}}$, and
2. Γ is $\mathrm{NP}_{\mathrm{search}}$-**hard**: as hard as any problem in $\mathrm{NP}_{\mathrm{search}}$.

To make "as hard as" precise: reductions.

Reductions: the picture (Chapter 4)



$T(n)$: time, each call = 1 step
$q(n)$: number of oracle calls
$h(n)$: size of each query y_i

Before (Ch 4): use Γ to solve Π. An algorithm for Γ in time T_Γ gives one for Π in time $T(n) + q(n) \cdot T_\Gamma(h(n))$.
Γ easy ⇒ Π easy.

Now: use the reduction's existence as evidence. Π known to be $\mathrm{NP}_{\mathrm{search}}$-hard.
Π hard ⇒ Γ hard: Γ is at least as hard as Π.

Polynomial-time reductions

Definition 19.8

$\Pi \leq_p \Gamma$ if there is a reduction from Π to Γ that runs in time $O(n^c)$ for a constant c, counting each oracle call as one step.

- So: polynomially many oracle calls, each of polynomial length (Lemma 19.9).

Lemma 19.10

Suppose $\Pi \leq_p \Gamma$.

1. If $\Gamma \in \mathrm{P}_{\mathrm{search}}$, then $\Pi \in \mathrm{P}_{\mathrm{search}}$.
2. If $\Pi \notin \mathrm{P}_{\mathrm{search}}$, then $\Gamma \notin \mathrm{P}_{\mathrm{search}}$.

Lemma 19.11. If $\Pi \leq_p \Gamma$ and $\Gamma \leq_p \Theta$, then $\Pi \leq_p \Theta$.

Reductions as evidence of hardness

Theorem 17.6: $\textsc{Graph Coloring} \leq_p \text{SAT}$.
In English: SAT is at least as hard as Graph Coloring. If Graph Coloring is $\text{NP}_{\text{search}}$-hard, this reduction shows SAT is too.

If $\textsc{Graph Coloring} \notin \text{P}_{\text{search}}$, then $\text{SAT} \notin \text{P}_{\text{search}}$.

Direction matters. To show Γ is hard, reduce a hard problem *to* Γ.

Not polynomial: $3\text{-}\textsc{Coloring} \to 2\text{-}\textsc{Coloring}$ (Theorem 14.2) runs in time $O(1.89^n)$. It can't be made polynomial unless $3\text{-}\textsc{Coloring} \in \text{P}_{\text{search}}$.

$\mathrm{NP}_{\mathrm{search}}$-completeness, formally

Definition 20.1

Γ is $\mathrm{NP}_{\mathrm{search}}$-**complete** if

1. $\Gamma \in \mathrm{NP}_{\mathrm{search}}$, and
2. Γ is $\mathrm{NP}_{\mathrm{search}}$-**hard**: $\Pi \leq_p \Gamma$ for every $\Pi \in \mathrm{NP}_{\mathrm{search}}$.

Proposition 20.2. If Γ is $\mathrm{NP}_{\mathrm{search}}$-complete: $\Gamma \in \mathrm{P}_{\mathrm{search}} \iff \mathrm{NP}_{\mathrm{search}} \subseteq \mathrm{P}_{\mathrm{search}}$.

To show Γ is $\mathrm{NP}_{\mathrm{search}}$-hard: reduce one known $\mathrm{NP}_{\mathrm{search}}$-hard problem to Γ (Lemma 19.11).
Next time: SAT is $\mathrm{NP}_{\mathrm{search}}$-complete, and the web of reductions.

Takeaway

- Classes contain *problems*. An algorithm is evidence about a problem.
- $\mathsf{P}_{\text{search}} \subsetneq \mathsf{EXP}_{\text{search}}$ is proven.
- $\mathsf{NP}_{\text{search}}$: short, checkable solutions. $\mathsf{NP}_{\text{search}} \subseteq \mathsf{EXP}_{\text{search}}$.
- For SAT, 3-$\textsc{Coloring}$, $\textsc{Long Path}$: in $\mathsf{P}_{\text{search}}$? Open.
- Reductions transfer hardness: from the problem you reduce *from* to the problem you reduce *to*.